Technical Test – AI Developer: E-commerce Market Analysis Agent

Objective

This test is designed to highlight your expertise in developing intelligent agents using language models and tool orchestration. It aims to evaluate your ability to design and implement complex agentic AI systems.

We consider this exercise as the starting point for a discussion. The quality of your reasoning and the clarity of your justifications are more important than the number of features implemented.

Guiding Principles

- **Focus on orchestration**:
  The primary objective is to evaluate your ability to make tools collaborate through an agent. The perfection of each individual tool is secondary.
- **Clarity over complexity**:
  A simple, well-explained, and robust solution is preferable to a complex but unfinished one.
- **Justify your choices**:
  The README.md file is just as important as the code. It is your space to explain your reasoning.

Context and Scenario

We are developing a market intelligence system for our e-commerce clients. The goal is to provide them with automated and personalized market analyses based on real-time data.

We have identified the need for an intelligent agent capable of orchestrating multiple analysis tools to produce comprehensive strategic reports on specific products or markets.

For this project, you will develop a **market analysis agent** that uses several specialized tools to:

- collect product data,
- analyze competition,
- assess customer sentiment,

- generate business recommendations.

Important

- Create a new Git repository for your solution
- A README.md file must be provided to:
- guide the user through installation and usage (steps 1 to 3)
- answer the theoretical questions (steps 4 to 7)
- Limit your programming work to **steps 1 to 3 only**
- Your code must be **executable and demonstrable**

Test Steps

1. Setup and Base Architecture

Create the foundational architecture of your market analysis agent:

- **Framework approach**: Choose and configure an agent framework (LangGraph, CrewAI, Google ADK, or others)

OR

- **Native approach**: Implement your own orchestration system directly in Python
- Implement the main orchestrator agent
- Define the REST API interface to receive analysis requests
- Create a modular structure for the different tools
- Containerize your solution using Docker for easier deployment and evaluation

Note: Both approaches are accepted. Justify your choice in the documentation.

API Design: You are free to design the API interface you find most appropriate for this use case.

2. Implementation of Specialized Tools

Develop **at least 3 tools** from the following list (you may use mock APIs or simulated data):

Suggested Tools:

- **Web Scraper Tool**: Collects prices and product information from different platforms
- **Sentiment Analyzer Tool**: Analyzes customer reviews and extracts insights

- **Market Trend Analyzer Tool**: Analyzes price and popularity trends
- **Report Generator Tool**: Compiles data into a structured report with visualizations

Quality Expectations:

The implementation must meet production standards with:

- modular architecture,
- proper error handling,
- ease of maintenance and debugging.

3. Testing and Validation

Implement unit tests covering:

- Functionality of individual tools
- Orchestration of the main agent
- Error case handling
- Output validation

Choose essential tests wisely without excessive coverage.

4. Data Architecture and Storage

Describe the data schema you would use to:

- Store analysis results
- Maintain request history
- Cache collected data
- Manage agent configurations

Which storage systems would you recommend and why?
(databases, cache, queues, etc.)

5. Monitoring and Observability

The client wants to monitor the health of the agent system in production. Explain your approach to:

- Tracing agent execution
- Collecting performance metrics

- Alerting in case of failure
- Measuring output quality

Which key metrics would you monitor?

6. Scaling and Optimization

Draw and/or explain how you would:

- Handle traffic spikes (100+ simultaneous analyses)
- Optimize LLM usage costs
- Implement an intelligent caching system
- Parallelize analysis tasks

7. Continuous Improvement and A/B Testing

Explain how you would:

- Automatically evaluate analysis quality (LLM as Judge)
- Compare different prompt engineering strategies
- Implement a user feedback loop
- Evolve agent capabilities over time

Evaluation Criteria

Agent Architecture (25%)

- Justified choice between existing framework or native implementation
- Tool orchestration design patterns
- Clear separation of responsibilities

Technical Quality (25%)

- Clean and maintainable Python code
- Robust error handling
- Appropriate testing and coverage

LLM Integration (25%)

- Efficient use of language models

- Prompt engineering per task
- Context and memory management

Innovation and Extensibility (25%)

- Advanced features implemented
- Extensible architecture
- Attention to UX/DX details

Expected Deliverables

- Source code with clear architecture and documentation
- Detailed README with installation and usage instructions
- Functional API with request examples
- Containerized solution (Dockerfile + docker-compose if needed)
- Example report generated by the system
- Documented answers to theoretical questions (steps 4–7)

Tips and Tricks

Estimated duration: **4–6 hours** depending on experience
Recommended Python version: **3.13**

- You may use mocked data to avoid API limitations
- Focus on agent orchestration rather than tool perfection
- Document your technical choices and justifications
- Your code **must be executable**
- Feel free to use free or low-cost services
  (OpenAI trial, DeepSeek, OpenRouter, Groq, etc.)

FAQ

Which agent framework should I use?

You may choose between:

- **Existing frameworks**: LangGraph, CrewAI, Google ADK, AutoGen, etc.
- **Native implementation**: Build your own orchestration system in Python

The key is to justify your choice and demonstrate mastery of agent orchestration.
A well-designed native solution can be just as valuable as a well-known framework.

How should paid APIs be handled?

Options include:

- Free or low-cost services: DeepSeek, OpenRouter, Groq, local Ollama, OpenAI/Anthropic trials
- Mock APIs or simulated data

The goal is to demonstrate architecture and patterns—not to spend money.

How should the system be tested?

Create concrete examples using real products (iPhone, Nike Air Max, etc.) and show generated reports. Simulated data is acceptable if needed.

Is production deployment required?

No. A local demonstration is sufficient.
The solution must be containerized with Docker to facilitate evaluation and ensure reproducibility.